

Search

Home

GitHub

CLASSES

ErrorBoundary

EVENTS

SIGINT

unhandledRejection

NAMESPACES

apiConfig

contactoEmergenciaService

diarioService

GLOBAL

404_Error_Handler

ACTIVIDADES

API_URL

ASPECTOS_COGNITIVOS

ActionCard

ActivitySelector

App

AuthButtons

AuthLayout

Button

CORS_Configuration

Card

Carousel

Checkbox

CircularProgress

CognitionSelector

Diario

DiaryBanner

DiaryEditor

DiaryEntry

EMOCIONES

EmergencyButton

EmergencyModal

EmotionSelector

EmotionStats

Footer

GFT /

frontend/src/components/molecules/Carousel.jsx

```
1.  /**
2.   * @file Componente de carrusel.
3.   * @description Muestra una colección de tarjetas en un carrusel nav
4.   * @requires react
5.   * @requires prop-types
6.   * @requires ../atoms/Card
7.   * @requires ../../styles/molecules/Carousel.css
8.   */
9.  import React, { useState } from "react";
10. import PropTypes from "prop-types";
11. import Card from "../atoms/Card";
12. import "../../styles/molecules/Carousel.css";
13.
14. /**
15.  * @function Carousel
16.  * @description Renderiza un carrusel de tarjetas con controles de n
17.  * @param {object} props - Las propiedades del componente.
18.  * @param {Array<object>} props.items - Una lista de objetos, cada u
19.  * @returns {JSX.Element} El componente de carrusel.
20.  */
21. const Carousel = ({ items }) => {
22.   const [currentIndex, setCurrentIndex] = useState(0);
23.   const itemsPerPage = 2;
24.
25.   /**
26.    * @function goToPrevious
27.    * @description Navega a la diapositiva anterior en el carrusel.
28.    */
29.   const goToPrevious = () => {
30.     setCurrentIndex((prevIndex) =>
31.       prevIndex === 0 ? Math.max(0, items.length - itemsPerPage
32.     );
33.   };
34.
35.   /**
36.    * @function goToNext
37.    * @description Navega a la siguiente diapositiva en el carrusel
38.    */
39.   const goToNext = () => {
40.     setCurrentIndex((prevIndex) =>
41.       prevIndex >= items.length - itemsPerPage ? 0 : prevIndex
42.     );
43.   };
44.
45.   if (!items || items.length === 0) {
46.     return <div className="carousel carousel--empty">No hay artí
47.   }
48.
49.   const visibleItems = items.slice(currentIndex, currentIndex + it
50.
51.   return (
52.     <div className="carousel">
```

```

53.         <button
54.             className="carousel__button carousel__button--prev"
55.             onClick={goToPrevious}
56.             aria-label="Anterior"
57.         >
58.             <
59.         </button>
60.
61.         <div className="carousel__content">
62.             {visibleItems.map((item, index) => (
63.                 <Card
64.                     key={currentIndex + index}
65.                     title={item.title}
66.                     description={item.description}
67.                 />
68.             ))}
69.         </div>
70.
71.         <button
72.             className="carousel__button carousel__button--next"
73.             onClick={goToNext}
74.             aria-label="Siguiente"
75.         >
76.             >
77.         </button>
78.
79.         <div className="carousel__indicators">
80.             {items.map((_, index) => (
81.                 <button
82.                     key={index}
83.                     className={`carousel__indicator ${
84.                         index === currentIndex ? 'carousel__indi
85.                     }`}
86.                     onClick={() => setCurrentIndex(index)}
87.                     aria-label={`Ir al artículo ${index + 1}`}
88.                 />
89.             ))}
90.         </div>
91.     </div>
92. );
93. };
94.
95. Carousel.propTypes = {
96.     /** Una lista de objetos, cada uno con `title` y `description` p
97.     items: PropTypes.arrayOf(
98.         PropTypes.shape({
99.             /** El título de la tarjeta. */
100.            title: PropTypes.string.isRequired,
101.            /** La descripción de la tarjeta. */
102.            description: PropTypes.string.isRequired
103.        })
104.    ).isRequired
105. };
106.
107. export default Carousel;

```